

Collection Data Structures Audit



July 17, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	6
Low Severity	8
L-01 sorted_set Documentation Recommends a Drain Path That Does Not Compile for Non-drop Keys	8
Notes & Additional Information	8
N-01 Undeclared Ability Bounds on Macros Surface as Opaque Post-Expansion Errors	8
N-02 Documentation Inaccuracies	9
N-03 Inconsistent Conventions	11
N-04 Pagination Documentation Overstates the Tiling Guarantee Under Concurrent Mutation	11
N-05 Macro Arguments Are Evaluated Out of Declaration Order	12
Conclusion	13
Appendix	14
Issue Classification	14

Summary

Type	Library	Total Issues	6 (6 resolved)
Timeline	From 2026-07-10 To 2026-07-15	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (1 resolved)
		Notes & Additional Information	5 (5 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin audited the [OpenZeppelin/contracts-sui](#) repository at the [63e1572](#) commit.

In scope were the following files:

```
collections
└─ sources
    ├── sorted_map.move
    └─ sorted_set.move
```

System Overview

The `openzeppelin_collections` package is part of OpenZeppelin Contracts for Sui and provides ordered collection types for Sui Move. The audited code comprises two modules: `sorted_map`, which implements `SortedMap<K, V>`, an ordered key-to-value map, and `sorted_set`, which implements `SortedSet<K>`, an ordered set of unique keys built as a thin wrapper over a map with an empty marker value. Both types extend the point-lookup functionality of the Sui framework's `vec_map` and `vec_set` with reads in key order, including head and tail access, floor and ceiling queries, nearest-key navigation, and sorted pagination.

Both collections are UID-less value types with no dynamic fields. Every entry lives inline in a single sorted vector, so each operation loads exactly one stored object, and capacity is bounded by Sui's object size limit rather than by any per-transaction dynamic-field access cap. Lookup is a binary search running in logarithmic time, while insertions and removals are linear due to vector shifting. A bare collection has no `key` ability, so integrators embed it as a field of their own object and expose their own entry functions around the library's API, which determines whether the collection is owned or shared.

Move has no storable function values and no standard semantic ordering trait, so neither collection stores its ordering. Instead, order is defined per call: the `_by` macro variants accept a caller-supplied strict less-than comparator, while the bare variants compile only for integer keys and use the built-in `<` operator. Much of the API is implemented as public Move 2024 macros, which forces several internal helpers, such as the position-based writers of `sorted_map` and the inner-map accessors of `sorted_set`, to be `public` even though they are not part of the supported API. The map additionally conserves resource values: replacing an entry returns the displaced value instead of dropping it, removals move values out, and only an empty collection can be destroyed, so store-only values such as `Coin` cannot be silently lost through the supported API.

The engagement covered two modules, `sorted_map` and `sorted_set`. Six example modules were additionally reviewed at a high level, as references for the intended integration patterns rather than as audited code: `order_book`, `prize_vault`, and `tick_registry` under the collections package's `examples/sorted_map` directory, and `allowlist`, `unlock_queue`, and `validator_set` under its `examples/sorted_set` directory. All six are out of scope, and each is explicitly marked as not production ready.

Security Model and Trust Assumptions

The library defines no privileged roles, capabilities, events, or global state, and it never inspects the transaction sender: every function and macro operates on whatever collection value the caller can reference. All access control, event emission, and ownership semantics are therefore the responsibility of the integrating module, which gates its own entry functions around the library's API. In particular, any party able to obtain a mutable reference to the wrapping object can insert, remove, or drain entries, including stored resource values such as `Coin`, so integrators must gate their entry functions accordingly. Moreover, all keys and values are world-readable on-chain regardless of the wrapper's ownership model, so no confidentiality can be assumed.

The central trust assumption is the comparator contract. Callers of the `_by` macros must supply a strict total order and must thread the same comparator to every call on a given collection. The library stores no comparator and cannot detect a violation, and the failure mode is silent: a non-strict or inconsistent comparator corrupts sorted order, producing duplicate inserts, missed removals, and wrong membership answers. For the map, this can leave values stranded at unreachable positions, whereas, for the set, the corruption is order-only and no key is ever lost. A coarse, non-injective comparator additionally collapses byte-distinct keys into a single entry under last-write-wins semantics, so which byte-variant is retained is only well-defined under an injective comparator. The comparator is further assumed to be pure and deterministic, as it may be evaluated multiple times within a single operation. In addition, since the bare macro variants hard-wire the integer `<` operator, they must never be mixed with `_by` calls that supply a different order on the same collection. The test-only `is_well_formed` oracle validates a collection against a caller-supplied comparator and therefore cannot detect a bug in that same comparator.

Integrators are likewise trusted to stay on the macro API: the forced-public internals, such as `insert_at`, `remove_at`, `inner_mut`, and `entries`, perform no ordering checks, and calling them directly can corrupt a collection. Furthermore, several operational boundaries are assumed to be managed by integrators: a collection whose key or value type lacks the `drop` ability must be drained and then destroyed through `destroy_empty`, capacity must be budgeted against Sui's object size limit (a protocol-configured limit for which the documented entry counts are illustrative measurements rather than guarantees, with no library-level capacity guard beyond the runtime's own abort), and the on-chain struct layouts are frozen at

first publish, with any future layout change intended to ship as a parallel type with consumer-driven migration rather than an in-place upgrade. Since writes are linear in collection size, permissionless writers on a shared wrapper can also grief it by bloating it toward the size limit. As such, integrators are expected to bound the number of entries, prefer tail-pop drains and paginated reads for large collections, and anticipate per-object rather than per-key write serialization when the wrapper is shared.

Several additional behavioral contracts are also assumed to be understood by integrators. The collections' ability requirements materialize jointly from the key and value types, and some are undeclared and only surface at compile time: a resource-valued map is store-only and must be drained and then destroyed through `destroy_empty`, a key type without the `copy` ability leaves `head`, `tail`, `keys`, and the other navigation entry points uncallable, and a key type without the `drop` ability leaves `upsert!` and `remove!` uncallable. The map returns every displaced or removed value (`upsert!` returns an `Option`, `remove!` returns the removed value, and the `pop_*` variants return the full entry), which callers are trusted to consume rather than discard.

The `from_sorted_*` constructors trust that their input is genuinely sorted under the same comparator used for all subsequent calls, aborting when an adjacent out-of-order pair is detected. For adjacent duplicate pairs, the map conserves values by aborting, whereas the set silently de-duplicates. Keys are immutable once inserted and can only be changed by removing and reinserting, navigation is defined in comparator order rather than numeric order, and paginated reads are only guaranteed to tile the key space when the collection is not mutated between pages. Similarly, `upsert` and `from_keys` intentionally diverge from the `vec_map` and `vec_set` semantics by performing a total upsert and de-duplicating, respectively, so those semantics must not be assumed.

When diagnosing failures, all library aborts fire at the library's own location with stable, append-only error codes, whereas aborts originating outside the library, such as the `std::vector` out-of-bounds abort of the position-based internals, `MoveObjectTooBig`, or arithmetic overflow in caller-supplied code, carry no library error code.

Low Severity

L-01 `sorted_set` Documentation Recommends a Drain Path That Does Not Compile for Non-`drop` Keys

A `SortedSet` whose key type lacks the `drop` ability cannot fall out of scope and must be emptied before being destroyed. Both the [module header](#) and the [destroy_empty doc comment](#) of `sorted_set.move` instruct integrators to drain such a set via `remove!` / `pop_*`. However, the `remove` family delegates to `sorted_map`, whose implementation discards the stored key and therefore requires the key type to have the `drop` ability. For the exact resource-key case the documentation targets, `remove!` does not compile, and the resulting error surfaces inside the library macro body rather than at the call site. Only `pop_front` and `pop_back`, which return the key by value, can drain such a set.

Consider updating both documentation locations to name `pop_front` / `pop_back` as the drain path for non-`drop` keys. Furthermore, consider declaring an explicit `drop` ability bound on the `remove` macros, mirroring `upsert_by`, so that any misuse is reported at the caller's type parameter instead.

Update: Resolved in [pull request #454](#).

Notes & Additional Information

N-01 Undeclared Ability Bounds on Macros Surface as Opaque Post-Expansion Errors

Several macros in `sorted_map.move` and `sorted_set.move` impose ability requirements on the key type that their signatures do not declare: the `remove` family requires `drop` (the stored key is discarded on removal), the set's `from_keys` family requires `drop` (it is built on `upsert_by`), and `from_sorted_keys_by` requires `copy` and `drop`. Likewise, the

navigation and pagination macros of both modules (`find_next_by`, `find_prev_by`, `next_key_by`, `prev_key_by`, and `keys_from_by`, along with their bare forms) require `copy`, as every returned key is dereferenced out of the backing vector. In contrast, `upsert_by` declares its `$K: drop` bound explicitly. When a consumer instantiates one of the undeclared macros with an incompatible key type, the compiler still rejects the code, so this is not a soundness issue. However, the resulting error points at an implementation detail deep inside the expanded macro body instead of stating the required ability against the caller's type parameter, leaving integrators to reverse-engineer whether the requirement is part of the API contract or an implementation accident.

Consider declaring the minimal ability bounds on each macro's type parameter, mirroring `upsert_by`, and documenting the requirement in each macro's doc comment so that misuse is reported as an explicit constraint violation.

Update: Resolved in [pull request #454](#).

N-02 Documentation Inaccuracies

The following documentation statements do not match the shipped code:

- The [note on the `upsert` divergence](#) in `sorted_set.move` attributes an aborting `upsert` to `sui::vec_set`, which has no `upsert`. The function that aborts on a duplicate is `vec_set::insert`.
- The [complexity section of the same module header](#) describes `from_keys!` as quadratic because of "a search per key". The search is logarithmic; the quadratic term comes from the linear vector shift each insert performs.
- The pagination documentation of `sorted_map.move` and `sorted_set.move` state unconditionally that successive pages have no overlap and no gap, with the map adding that concatenating them "reconstructs the tail exactly". This holds only when the collection is not mutated between pages; interleaved writes can drop or repeat keys across pages.
- The abort documentation of `insert_at` and `remove_at` describes an out-of-bounds index as a "Native out-of-bounds abort inside `std::vector`", and a [comment in `pop\_front`](#) repeats the claim for `vector::remove`. However, `vector::insert` and `vector::remove` are regular Move functions that abort with the `EINDEX_OUT_OF_BOUNDS` code (`0x20000`) at `location = std::vector`, an ordinary, matchable Move abort rather than a native error. The parallel statements on `value_at`, `value_at_mut`, and in `pop_back` are accurate, as `vector::borrow`, `vector::borrow_mut`, and `vector::pop_back` are native functions. The

distinction determines how a consumer pins an `#[expected_failure]` test, which the surrounding documentation otherwise treats with care.

- The [comparator rationale](#) in the `sorted_map` module header states that "`std::compare` orders by BCS bytes, not semantic value". No `compare` module exists in the Move standard library the package builds against; the referenced module was never published on Sui.
- The [from_keys_by documentation](#) states that the inserts are driven "via a `do!` loop body", whereas the [implementation](#) uses `fold!`.
- The [SortedMap abort list in the README](#) opens with "Only these abort" and names `remove / remove_by` yet only the bare `borrow / borrow_mut`, although `borrow_by!` and `borrow_mut_by!` abort with `EKeyNotFound` identically.
- The [resource-conservation note in the README](#) contains the garbled phrase "`upsert`'s upsert returns the displaced value".
- The [SortedSet security note in the README](#) states that after a comparator violation "every key is still physically present and recoverable via `keys()`". Under a coarse (non-injective) comparator, which [the same README](#) warns "silently collapses byte-distinct keys", an upsert on a compare-equal key physically drops the stored key and retains the incoming one, so the claim holds only for injective comparators.
- The [SortedSet usage snippet in the README](#) returns a key-only `Watchlist` object from `new` while providing no transfer, share, or keep function. Since the object lacks `store` and `drop`, a programmable transaction block cannot consume the returned value, making the snippet unusable as written.
- The fields of the [Entry struct](#) lack the field-level documentation comments the project style guide requires for public types.

None of these statements affects the behavior of deployed code. However, the library is explicitly positioned as an integration reference, so inaccurate doc comments can propagate directly into consumer assumptions.

Consider correcting each inaccurate statement and adding the missing field documentation for `Entry` to align the documentation with the implemented behavior and project style guidance.

Update: Resolved in pull requests [#492](#) and [#454](#).

N-03 Inconsistent Conventions

Two cosmetic deviations from the library's own conventions exist in [sorted_map.move](#):

- The bare [next_key](#) and [prev_key](#) macros call [find_next_by!](#) / [find_prev_by!](#) directly instead of delegating to their own [_by](#) forms, as every other bare macro in the module does.
- Two macro bodies ([borrow_by](#) and [upsert_by](#)) invoke [search!](#) in receiver form ([map.search!\(...\)](#)), while the seven remaining call sites use the free-function form ([search!\(map, ...\)](#)).

Consider aligning both patterns for readability.

Update: Resolved in pull requests [#493](#) and [#454](#).

N-04 Pagination Documentation Overstates the Tiling Guarantee Under Concurrent Mutation

The [keys_from_by! documentation](#) states that resuming pagination by passing the last returned key back as [from](#) with [include == false](#) yields pages with "no overlap and no gap, so concatenating them reconstructs the tail exactly". The claim is repeated by the [set variant](#) and the README ([line 79](#) and [line 135](#)).

However, each call recomputes the resume position against the current entries, so the guarantee only holds while the collection is unchanged. This is always the case within a single non-mutating transaction, but not for a cursor persisted across transactions against a shared object. In that setting, pagination has standard keyset semantics: keys inserted at or before the cursor between reads never appear in any later page, and an empty page means "no keys past the cursor at this moment" rather than "iteration complete". Consumers that treat paged reads as an exhaustive scan can therefore silently miss entries.

Consider qualifying the tiling guarantee at all three documentation sites, and explicitly stating the keyset semantics of cursors persisted across transactions.

Update: Resolved in [pull request #492](#).

N-05 Macro Arguments Are Evaluated Out of Declaration Order

In `find_next_by!`, `find_prev_by!`, and `keys_from_by!` of `sorted_map.move`, the macro body binds `$include` (and `$limit`) before evaluating `$key` or `$from`, so caller-supplied arguments run out of declaration order. The `sorted_set` wrappers and every bare variant inherit this behavior. Results are unaffected for pure arguments. However, side-effecting or aborting argument expressions execute in an order that the macro signatures do not reveal.

Consider binding every macro parameter to a local at the top of each macro body, in declaration order, mirroring `add_by` and `upsert_by`.

Update: Resolved in [pull request #495](#).

Conclusion

The `openzeppelin_collections` package provides two ordered collection types for Sui Move, `SortedMap` and `SortedSet`, which keep their entries sorted under a caller-supplied comparator inside a single inline vector and extend the Sui framework's `vec_map` and `vec_set` with ordered navigation and pagination. OpenZeppelin audited the two library modules.

The core data-structure logic was found to be carefully constructed, with resource-conserving write paths and aborts routed to the library's own locations. The issues identified during the engagement concentrate on the integration surface rather than on the sorting or lookup logic itself. Several documentation statements diverge from the implemented behavior, including the recommended drain path for sets with non-droppable keys and the pagination guarantees under concurrent mutation. In addition, several macros impose ability requirements on their type parameters that their signatures do not declare, surfacing misuse as opaque post-expansion compiler errors.

Since the library stores no comparator and performs no caller checks, its guarantees are conditional on integrator discipline: threading one strict total order consistently, staying on the macro API rather than the forced-public internals, and budgeting for the object size ceiling. The in-code documentation states these obligations prominently and is positioned as an integration reference. Keeping it exactly aligned with the shipped behavior is, therefore, the main area for continued attention.

The OpenZeppelin Contracts for Sui development team is thanked for the thorough in-code documentation and the clearly stated design rationale that accompanied the codebase throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.